

Interview Preparation

Set 1

Q1 What is the optimization equation of GBDT?

Input: Training set $\{(x_i, y_i)\}_{i=1}^n$, a differentiable loss function $L(y, F(x))$, a differentiable number of iterations M .

Algorithm: Initialize model with a constant value:

1.

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$$

2. For $m=1$ to M :

1. Compute so-called pseudo-residuals:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$

for $i=1$ to n .

2. Fit a base learner (or a weak learner, eg tree) closed under scaling, $h_m(x)$ to pseudo-residuals in train set using the training set $\{(x_i, r_{im})\}_{i=1}^n$

3. Compute multiplier γ_m by solving the following one-dimensional optimization problem:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$$

4. Update the model :

$$F_m(x) = F_{m-1}(x) + \gamma h_m(x).$$

3. Output $F_M(x)$

Q2 write the formulation of hinge loss.

$$l(y) = \max(0, 1 - t \cdot y)$$

Q3 what is the train time complexity of KNN?
 $O(nd)$ or $O(1)$

Q4 what is the test time complexity of KNN in brute force?
 $O(knd)$

Q5 what is the test time complexity of KNN if we use kd-tree?
 $O(n)$

Q6 How will you regularize the KNN model?
We can regularize the KNN model in practice by optimizing the value of k in practice.

There are some advanced mathematical concepts that can be used but they are not implemented in practice.

Q7 Which of these models are preferable when we have low complexity power?

- a. SVM
- b. KNN
- c. Linear Regression
- d. XGBoost ✓

Q8 What is Laplace Smoothing?

In text classification, if a word is not present in training data, for test data may have a word whose probability may turn out to be zero.

At that point of time we do Laplace Smoothing. If $\alpha = 1$, it is called add-on Smoothing. Laplace Smoothing ~~is done~~ if done, is done on all the words not just one word.

Q9 How will you regularize your naive bayes model?

Laplace Smoothing

Q10 Can we solve dimensionality reduction with SGD?

Ans We can implement PCA using Gradient descent. We can pose PCA as a variance maximization problem. These are some hints.

The objective is to find out the direction in which the variance, $E(\vec{x}\vec{x}^T)$ is max. Let $\vec{w}$ denote the unit vector, direction along which the variance is maximum. The variance along this direction is given by:

$$\begin{aligned}\sigma_{\vec{w}}^2 &= \frac{1}{n} \sum (\vec{x}_i \cdot \vec{w})^2 \\ &= \frac{1}{n} (\mathbf{X}\mathbf{w})^T (\mathbf{X}\mathbf{w}) \\ &= \frac{1}{n} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} \\ &= \mathbf{w}^T \underbrace{\frac{\mathbf{X}^T \mathbf{X}}{n}}_{\mathbf{V}} \mathbf{w} \\ &= \mathbf{w}^T \mathbf{V} \mathbf{w}\end{aligned}$$

We can take gradients w.r.t. to $\vec{w}$ and set it zero to find the optimum values.

$$\begin{aligned}\mathcal{L}(\mathbf{w}, \lambda) &\equiv \sigma_{\vec{w}}^2 - \lambda (\mathbf{w}^T \mathbf{w} - 1) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= \mathbf{w}^T \mathbf{V} - \lambda \mathbf{w} \\ \frac{\partial \mathcal{L}}{\partial \lambda} &= \mathbf{w}^T \mathbf{w} - 1\end{aligned}$$

Here $\mathcal{L}$ is the modified objective with Lagrange multiplier.

Set 2

Q11 Which one will be doing more computations GD or SGD?

In both gradient descent (GD) and Stochastic gradient descent (SGD) you update a set of parameters in an iterative manner to minimize an error function.

While in GD, you have to run through ALL the samples in your training set to do a single update for a parameter in a particular iteration, in SGD, on the other hand, you use only ONE or SUBSET of training sample from your training set to do the update for a parameter in a particular iteration. If you use a SUBSET, it is called a Minibatch Stochastic Gradient Descent.

Thus, if the number of training samples are large, in fact very large, then using gradient descent may take too long because in every iteration when you are updating the value of the parameters, you are running through the complete training set. On the other hand, using SGD will be faster because you use only one training sample and it starts improving itself right away from the first sample.

(they are required to ensure $\vec{w}$ is a Unit vector.

Setting the derivatives to zero, gives you an eigenvalue problem.

If one wants to solve it using Gradient Descent, because we have obtained the gradients here, we can solve minimizing L using gradient descent, as well.

~~Update~~

SGD often converges much faster compared to GD but the error function is not well minimized as in the case of GD. Often in most cases, the close approximation that you get in SGD for the parameter values are enough because they reach the optimal values and keep oscillating there.

Q12 If A is a matrix of size $(3,3)$ and B is a matrix of size $(3,4)$ how many number of multiplications that can happen in the operations $A * B$?

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

Mismatch between no. of rows and column in multiplication.

Q13 What is the optimization equation of Logistic Regression?

$$W^* = \underset{W}{\operatorname{argmin}} \sum_{i=1}^n \log(1 + \exp(-y_i W^T x_i))$$

In this formulation we assumed y_i to be either $+1$ or -1 but not 1 or 0 .

Q15 Write the code for proportional sampling.

Proportional sampling is the method of picking an element proportional to its weight, i.e., the higher the weight of the object, the better are its chances of being selected.

//code
import random

Step Sum = $A[0] + A[1] + \dots + A[n]$

where, n = index of the last element of your defined input

//code

$A = [1, 2, 3, 4, 5, 6]$

$C = \{ \}$

$D = \{ \}$

$B = []$

def pick_a_no_from_list(A):

Step 1: Compute the total sum

Sum = 0

Sum1 = 0

for i in range(len(A)):

Sum += A[i]

Step 2 Calculate the normalized sum of each element in the list of inputs.

$$A_0' = \frac{A[0]}{\text{Sum}}; \quad \text{Ⓢ}$$

$$A_1' = \frac{A[1]}{\text{Sum}} \dots$$

where A' represents the normalized sum of each element

// code

for j in range(len(A)):

$$c["_dash\{\}\".format(j)] = A[j]/\text{sum}$$

Step 3 calculate the cumulative sum of each element using all the normalized values.

$$A_0 \sim = A_0';$$

$$A_1 \sim = A_0 + A_1'; \dots$$

//code

```
for k in range(len(A)):
```

```
    sum1 += c["d_dash {}".format(k)]
    d["d_cum {}".format(k)] = sum1
```

Step 4 Pick a random value in the range of (0,1).

```
r = random.uniform(0.0, 1.0)
```

```
if r <= A0 ~ = Return choice-1
```

```
else if r <= A1 ~ = Return choice-2;
```

//code

```
r = random.uniform(0.0, 1.0)
```

```
for m in range(len(A)):
```

```
    if r <= d["d_cum {}".format(m)]:
```

```
        return(A[m])
```


Step 5 Final Implementation

//code

```
def sampling_based_on_magnitude():  
    for i in range(1, 100):  
        number = pick_a_number_from_list(A)  
        b.append(number)
```

```
sampling_based_on_magnitude()
```

```
print
```

```
c = (sorted(A, reverse = True))  
print(c)
```

```
for x in c:  
    print("count of { } is".format(x, b.count(x)))
```

Q14. How will you calculate the $P(X/Y=0)$ in case of Gaussian naïve Bayes?

An approach to overcome this 'zero frequency problem' in a Bayesian setting is to add one to the count for every attribute value-class combination when an attribute value doesn't occur with every class value. So, for example, say your training data looked like this:

	Spam = Yes	Spam = No
Time Zone = US	10	5
Time Zone = EU	0	0

$$P(\text{Timezone} = \text{US} | \text{Spam} = \text{Yes}) = \frac{10}{10 + 0} = 1$$

$$P(\text{Timezone} = \text{EU} | \text{Spam} = \text{Yes}) = \frac{0}{10} = 0$$

Then you should add one to every value in this table when you're using it to calculate probabilities:

	Spam = Yes	Spam = No
Timezone = US	11	6
Time Zone = EU	1	1

$$P(\text{Time Zone} = \text{US} | \text{Spam} = \text{Yes}) = \frac{11}{11 + 1} = \frac{11}{12}$$

$$P(\text{Timezone} = \text{EU} | \text{Spam} = \text{Yes}) = \frac{1}{12}$$

Q16 What are the hyperparameters in Kernel SVM?

The Support vector machine (SVM) is a very different approach for supervised learning than decision trees. The different Kernels of SVM stand as its various hyperparameters. It maps the observations into some feature space. Ideally the observations are more easily linearly separable after this transformation. There are multiple standard Kernels for this transformation eg Linear Kernel, Polynomial Kernel, and Radial Kernel.

The choice of the kernel and their hyperparameters affect greatly the separability of the classes in classification and the performance of the algorithm.

The linear kernel

The linear kernel is $K(x_i, x_j) = x_i^T x_j$. When using this kernel we only have one hyperparameter in SVM: the cost parameter C .

The polynomial kernel

The polynomial kernel is $K(x_i, x_j) = (r + \gamma x_i^T x_j)^d$.

Usually the parameter r is set to zero and γ to a fixed value, eg. $1/n$ with n being the number of observations. Besides the cost parameter C , the integer parameter d has to be tuned, usually values between 1 and 10 are chosen.

The radial kernel

The radial kernel is $K(x_i, x_j) = e^{-\gamma x_i^T x_j}$. This kernel is the most used and most successful kernel, due to the flexibility of separating observations with this method. Additionally to the cost parameter C , the hyperparameter γ has to be tuned. In the paper "A practical guide to

Ans 17. Linear classifier (SVM, Logistic Regression) with SGD training. This estimator implements regularized linear models with stochastic gradient descent (SGD) learning: the gradient of the loss is estimated (each sample at a time) and the model is updated along the way with a decreasing/strength schedule/learning rate. SGD allows minibatch learning rate via the partial fit method. For best results, using the ~~the~~ default learning rate schedule, the data should have zero mean and unit variance.

This implementation works with data represented as dense or sparse arrays of floating point values for the features. The model it fits can be controlled with the loss parameter; by default, it fits a linear support vector machine (SVM).

The regularizer is a penalty added to the loss function that shrinks model parameters towards the zero vector using either squared euclidean norm L_2 or the absolute norm L_1 or a combination of both (Elastic Net). If the parameter update crosses the 0.0 value because of the regularizer, the update is truncated to 0.0 to allow for learning sparse models and achieve online feature selection.

1) Support vector classifier. they recom
to search C in the range $(-\infty, \infty)$
 γ in the range $[2^{-5}, 2^3]$

Q17 What are the hyperparameters in SGD with hinge loss?

Q18 Is hinge loss differentiable if not how we will modify it so that you apply SGD.
Hinge loss is not differentiable and cannot be used with methods which are differentiable like stochastic gradient descent (SGD). In this case Cross Entropy (log loss) can be used. This function is convex like hinge loss and can be minimized using SGD.

Ans 19. Both Adam and RMSprop are gradient descent algorithm.

In Adam, while learning rate is a hyperparameter, BCMA is short for "bias corrected". g_j is the j^{th} component of the gradient and $\text{BCMA}(g_j)$ is a bias corrected moving average of the j^{th} components of the gradients that were calculated. Similarly, $\text{BCMA}(g_j^2)$ is a bias-corrected moving average of the squares of the j^{th} components of the gradients that were calculated. For each moving average, the decay factor / smoothing factor is a hyperparameter. Both Adam and Tensorflow use the notation: β is the decay factor for $\text{BCMA}(g_j)$, β^2 is the decay factor for $\text{BCMA}(g_j^2)$. The denominator is actually

$\sqrt{\text{BCMA}(g_j^2)} + \epsilon$, while ϵ is a small hyperparameter, but can be ignored for simplicity.

In RMSprop with momentum: $\Delta x^{(t)}$, momentum decay factor is a hyperparameter and it can be assumed that it is in $(0, 1)$. Usually Tensorflow, this is the momentum argument of RMSpropOptimizer . $g_j^{(t)}$ is the j^{th} component of the gradient in the t^{th} step. $\text{MA}(g_j^2)$ is a moving average of the square of the j^{th} components of the gradients that were calculated. The decay factor of this moving average is a hyperparameter and in Tensorflow, this is the decay argument of RMSpropOptimizer .

Q20. What is RMSPROP?

RMSprop is a gradient based optimization technique used in training neural networks. It was proposed by the father of back-propagation Geoffrey Hinton. Gradients of very complex functions like neural networks have a tendency to either vanish or explode as the data propagates through the function. RMSPROP was developed as a stochastic technique for minibatch learning.

Set 3

Q21. What is ADAM?

The Adam optimization algorithm is an extension to stochastic gradient descent that has recently seen broader adoption for deep learning applications in computer vision and natural language processing.

The authors describe Adam as combining the advantages of two other extensions of stochastic gradient descent. Specifically,

* Adaptive Gradient Algorithm (Ada Grad)

that maintains a per-parameter learning rate that improves performance on problems with sparse gradients.

* Root Mean Square Propagation (RMSprop)

that also maintains per-parameter

learning rates that are adapted based on the average of recent magnitudes of the gradient for the weight. This means the algorithm does well on online and non-stationary problems.

Adam realizes the benefits of both Adagrad and RMSprop.

Instead of adapting the parameter learning rates based on the average first moment as in RMSprop, Adam also makes use of the average of the second moments of the gradients.

Specifically, the algorithm calculates an exponential moving average first moment as in RMSprop.

Adam also makes use of the average of the second moments of the gradients.

Specifically, the algorithm calculates an exponential moving average of the gradient and the squared gradient, and the parameters β_1 and β_2 control the decay rates of these moving averages.

The initial values of the moving averages and beta 1 and beta 2 values close to 1.0 result in a bias of moment estimates towards zero. This bias is overcome by first calculating the biased estimates before then calculating bias-corrected estimates.

Q22 What is the maximum and minimum value of gradient of the sigmoid function?

The maximum value of the derivative of sigmoid is 0.25 and minimum is 0

The maximum value is less than 1.

Q23 What is RELU? Is it differentiable?

The rectified linear activation function or ReLU for short is a piecewise linear function that will output the input directly if it is positive, otherwise, it will output zero.

It has become the default activation function for many types of neural network because a model that uses it is easier to train and often achieves better performance.

ReLU is differentiable at all the points except 0. The left derivative at 0 is 0 and right derivative is 1.

Q24 what is F1 score? Q25 what is precision and recall?

In Statistical analysis of binary classification the F-score or F-measure is a measure of a test's accuracy. It is calculated from the precision and recall of the test, where the precision is the no. of true positive results divided by the no. of all positive results, (including those not identified correctly).

Recall is the no. of true positive results divided by the no. of all samples that should have been identified as positive.

F₁ score is the harmonic mean of the precision and recall.

Q26 Name a few weight initialization techniques?

Zero Initialization.

Random Initialization.

He initialization

Xavier "

Q27 which of these will have more numbers of tunable parameters?

a. $7 \times 7 \times 512 \rightarrow \text{Flatten} \rightarrow \text{Dense}(512)$

b. $7 \times 7 \times 512 \rightarrow \text{Conv}(512, (7, 7))$

a. $7 \times 7 \times 512 \times 512 = 20.7M$

b. $7 \times 7 \times 512 = 25K$ ($\because$ we are using Conv2d)

Q28. What is overfitting and underfitting?
Overfitting means good performance on training data and poor generalization to other data.

Underfitting means poor performance on the training data and poor generalization to other data.

Q29. What do you do if a deep learning model is overfitting?
Handling overfitting:

1. Reduce the network's capacity by removing layers or reducing the number of elements in the hidden layers.
2. Apply regularization, which comes down to adding a cost to the loss function for large weights.
3. Use dropout layers, which will randomly remove certain features by setting them to zero.

Q30. What is the batch normalization layer?
Batch normalization is a technique for training very deep neural networks that standardizes the inputs to a layer for each mini-batch.

This has the effect of stabilizing the learning process and dramatically

Set 4

Q31 Write keras code to add a BN layer in an existing network?

```
from keras.layers.normalization \
import BatchNormalization
```

```
model = Sequential()
model.add(Dense(64, input_dim=14, init='uniform'))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.5))
```

```
Sgd = SGD(lr=0.1, decay=1e-6, momentum=0.9,
restoration=True)
```

```
model.compile(loss='binary_crossentropy',
optimizer=Sgd)
```

```
model.fit(x_train, y_train, nb_epoch=
Batch_size=16, (do
show_accuracy=True,
validation_split=0.2,
verbose=2)
```

Q32 What are the no. of tunable parameters in BN layer?

In the below procedure to perform batch normalization, the tunable parameters are nothing but the parameters which are updated in the training stage.

These are gamma and beta in batch normalization.


```
def batch_norm(x, gamma, beta, moving_mean, moving_var, eps, momentum):
    if not autograd.is_training():
        x_hat = (x - moving_mean) / np.sqrt(moving_var + eps)
    else:
```

```
    assert len(x.shape) in (2, 4)
```

```
    if len(x.shape) == 2:
```

```
        mean = x.mean(axis=0)
```

```
        var = ((x - mean)**2).mean(axis=0)
```

```
    else:
```

```
        mean = x.mean(axis=(0, 2, 3), keepingdims=True)
```

```
        var = ((x - mean)**2).mean(axis=(0, 2, 3), keepingdims=True)
```

```
    x_hat = (x - mean) / np.sqrt(var + eps)
```

```
    moving_mean = momentum * moving_mean +
```

```
        (1.0 - momentum) * mean
```

```
    moving_var = momentum * moving_var +
```

```
        (1.0 - momentum) * var
```

```
    y = gamma * x_hat + beta
```

```
    return y, moving_mean, moving_var
```


Q33 What is convolution operation?

Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image.

The innovation of convolutional neural networks is the ability to automatically learn a large no. of filters in parallel specific to a training dataset under the constraints of a specific predictive modeling problem, such as image classification. The result is highly specific features that can be detected anywhere on input images.

Q34 No. of parameters in a convolutional neural network given in architecture. No. of parameters in a convolutional layer would be:

$$((m * n * d) + 1) * k$$

$$(((m * n * d) + 1) * k)$$

added 1 because of the bias term for each filter. The same expression can be written as follows:-

Ans 36) Precision Macro Avg =

$$\frac{Prec_1 + Prec_2 + Prec_3 + Prec_4 + Prec_5}{n}$$

Recall Macro Avg =

$$\frac{Recall_1 + Recall_2 + Recall_3 + Recall_4 + Recall_5}{n}$$

Since question is for 5 class,
the no. of classes would be 5 in the
formula.

$$((m^2 + 1) + (n^2 + 1))$$

$$((m^2 + 1) + (n^2 + 1))$$

(Shape of width of filter *
 Shape of height of the filter *
 no. of filter in the previous layer + 1) *
 no. of filters.

Q35 What are the inputs required to calculate neural network given in architecture.

Q35 What are the inputs required to calculate the average F1 score?

Computing F1-score can be done by adding TP, FP and FN from each fold, computing precision and recall and finally the F1-score.

Q36 What ^{is} macro average F1-score for 5 class classification problem.

Q37 How do you get probabilities for Random Forest classifier outputs.
 It's just the proportion of votes of the trees in the ensemble.

A Random Forest Classifier is a collection of Decision Tree Classifiers. No matter how ~~long~~ big training set, a decision tree simply returns a decision. One class has probability 1, the other has 0.

Random Forest simply votes among the results
`predict_proba()` returns the no. of votes

Ans 38) Not required. The logistic regression produces well calibrated probabilities since it directly minimizes the log loss.

Ans 39)

for each class divided by the no. of trees in the forest. Hence, your precision is exactly $1/n$ estimators.

If you want more precision, add more estimators. If you want to see variation at the 5th digit, you need $10^{5.5}$ (- or) estimators (100,000), which is excessive. You normally don't want more than 100 estimators, and often not that many.

Q38 Is the calibration classifier required to get probability values for logistic regression?

Q39 How does Kernel sum work in test time?

Q40 What kind of base learners are preferable in random forest classifiers? They are decision trees. Each tree must be independently trained.

Set 5

Q41 How does bootstrapping work in Random forest classifiers? When training, each tree in a random forest learns from a random sample of the data points. The samples are drawn with replacement, known as bootstrapping, which means that some samples will be used multiple times in a single tree.

Q42 Difference b/w one vs rest and one vs one?

One-vs-Rest for Multi-class Classification

One vs Rest is a heuristic method for using binary classification algorithms for multi class classification.

It involves splitting the multi-class dataset into multiple binary classification problems. A binary classifier is then trained on each binary classification problem and predictions are made using the model that is the most confident.

For classifying red, blue & green.
Binary classification problem 1:
red vs [blue, green].

Binary classification problem 2:
blue vs [red, green].

Binary classification problem 3:
green vs [red, blue].

Drawback: For this approach, it requires one model to be created for each class.

Although, this approach requires that

each model predicts a class membership probability or a prob-like score. The argmax of these scores is then used to predict a class.

eg - Logistic Regression, Perceptron.

One-vs-One for Multi-Class Classification

It is another heuristic method for using binary classification algorithm for multi class classification.

Like one-vs-Rest, one-vs-one splits a multi-class classification dataset into binary classification problems. Unlike one-vs-rest that splits it into one binary dataset for each class, the one-vs-one approach splits the dataset into one dataset for each class vs every other class.

Binary Classification Problem 1: red vs blue
Binary Classification Problem 2: red vs green
Binary Classification Problem 3: red vs yellow
Binary Classification Problem 4: blue vs green
Binary Classification Problem 5: blue vs yellow
Binary Classification Problem 6: green vs yellow

classically, this approach is suggested for support vector machines (SVM) and related kernel-based algorithms. This is believed because the performance of kernel methods does not scale in proportion to the size of the training dataset and using subsets of the training data may counter this effect.

Q43 Which one is better, one vs rest or one-vs-one?

The one-vs-one multi-class classification option only splits the primary dataset into a single binary classification for each pair of classes. Although the one-vs-rest approach cannot handle multiple datasets, it trains less no. of classifiers, making it a faster option and often preferred.

Q44 What will happen if gamma increases in RBF kernel SVM?

If gamma is large in RBF kernel SVM, it leads to wiggly decision boundary and causes overfitting.

Reason:

Larger RBF kernel bandwidth (ie smaller γ) produces smoother decision boundaries. They produce smoother feature space mappings.

Q45 Explain linear regression.
 Linear Regression attempts to model the relationship b/w 2 variables by fitting a linear equation to observed data.

A linear regression line has an equation of the form $Y = mx + C$, where X is explanatory variable and Y is the dependent variable.

Q46 What is the difference b/w one-hot encoding and a binary bag of words?
 One-hot encoding can be performed if a column has a few values. Whereas binary encoding can be useful where the cardinality of the column is higher.

For eg - 50 US states $\rightarrow$ Binary encoding creates fewer columns as compared to one-hot encoding. It is more efficient.

Q47) Kernel SVM and Linear SVM (SGD classifier with hinge loss). Which has low latency and why?
 Explanation through sklearn code:

```
from sklearn.svm import SVC
from sklearn.linear_model import SGDClassifier
from sklearn.svm import LinearSVC
from sklearn import datasets
import numpy as np
```